Introduction

Introduction and Revision: Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Introduction and Revision

Core Course:

4 hours of lecture + 2 hours of laboratory

5 Credits

One of the most important courses in the curricula

Pre-requisites:

1. Programming in C
2. Good knowledge of pointers, arrays and structures

Text Book:

"Data Structures using C / C++", Langsum Yedidyah, Moshe J Augenstein, Aaron M Tenenbaum Pearson Education Inc, 2nd edition, 2015.

© Confidential Information of PES University. All Rights Reserved - 2026

Introduction and Revision

Reference Book:

"Data Structures and Program Design in C", Robert Kruse, Bruce Leung, C.L Tondo, Shashi Mogalla, Pearson, 2nd Edition, 2019

Anchor for this Course:

N S Kumar

M.Tech (Computer Science IIT Madras), BE (Mech) UVCE

- Over 40 years of teaching experience
- Corporate trainer
- Visiting Faculty PESU for over 13 years

Ph: 98450 66845

© Confidential Information of PES University. All Rights Reserved - 2026

Assessment Scheme

ISA1:

1. After Unit 1 and Unit 2
2. 40 marks reduced to 20 marks
3. All objective questions
4. 10 one markers and 5 two markers per unit

ISA2:

5. After Unit 3 and Unit 4
2. 40 marks reduced to 20 marks
3. All objective questions
4. 10 one markers and 5 two markers per unit

© Confidential Information of PES University. All Rights Reserved - 2026

Assessment Scheme

ESA:

1. 100 marks reduced to 50 marks
2. 20 marks objective questions
3. 10 one markers and 5 two markers
4. 80 marks subjective questions; 20 marks from each unit

Project:

5. 10 marks
2. shall be conducted in the last week of the semester
3. 6 marks for active participation and submission
4. 4 marks for correctness

© Confidential Information of PES University. All Rights Reserved - 2026

Laboratory

1. One practice assignment
2. Ten programming assignments on various topics
3. Shall synchronize with theory
4. A few assignments in reading/debugging programs
5. Last week activity is part of project associated with theory

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Functions and Pointers

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Functions, Pointers and Arrays

1. Which of the following is false?

```
int a[10] = {1, 2, 3} ; int *p = a;
int foo(int *q) { return (int)q; }
```

a) sizeof(a) == sizeof(p)

b) *a == *p

c) a == p

d) foo(a) == foo(p)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Functions, Pointers and Arrays

1. Which of the following is false?

```
int a[10] = {1, 2, 3} ; int *p = a;
int foo(int *q) { return (int)q; }
```

a) sizeof(a) == sizeof(p)

b) *a == *p

c) a == p

d) foo(a) == foo(p)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Functions, Pointers and Arrays

2. Comment :

```
void disp(int* f, int* l) {
    while(f != l) {
        printf("%d\n", *f); ++f;
    }
}
```

int a[] = {1, 2, 3, 4, 5}; disp(a, a + 5);

a) Syntax error; size of the array not specified while evaluating

b) Runtime error; a + 5 is a dangling pointer

c) Perfectly correct; no error

d) Runtime error; dangling pointer has been dereferenced

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Functions, Pointers and Arrays

2. Comment :

```
void disp(int* f, int* l) {
    while(f != l) {
        printf("%d\n", *f); ++f;
    }
}
```

int a[] = {1, 2, 3, 4, 5}; disp(a, a + 5);

a) Syntax error; size of the array not specified while evaluating

b) Runtime error; a + 5 is a dangling pointer

c) Perfectly correct; no error

d) Runtime error; dangling pointer has been dereferenced

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Functions, Pointers and Arrays

3. Comment :

```
void what(int* x, int* y) {

    *x = *x ^ *y;

    *y = *x ^ *y;

    *x = *x ^ *y;

}
```

int a = 10; int b = 20; what(&a, &a); What will be the values of a and b?

a) 10 20

b) 20 10

c) 0 0

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Functions, Pointers and Arrays

3. Comment :

```
void what(int* x, int* y) {

    *x = *x ^ *y;

    *y = *x ^ *y;

    *x = *x ^ *y;

}
```

int a = 10; int b = 20; what(&a, &a); What will be the values of a and b?

a) 10 20

b) 20 10

c) 0 0

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Functions, Pointers and Arrays

4. Comment :

```
void what() {
    int *p;
    {
        int a = 10; p = &a;
    }
    printf("%d \n", *p);
}
```

a) prints 10

b) Creates a garbage

c) Creates a dangling reference and dereferences it

d) Creates aliases

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Functions, Pointers and Arrays

4. Comment :

```
void what() {
    int *p;
    {
        int a = 10; p = &a;
    }
    printf("%d \n", *p);
}
```

a) prints 10

b) Creates a garbage

c) Creates a dangling reference and dereferences it

d) Creates aliases

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Functions, Pointers and Arrays

5. Comment :

```
void what() {

    int *p = (int*)malloc(sizeof(int)); // assume malloc succeeds

    *p = 111;

}
```

a) No error; memory reclaimed when the function terminates

b) Creates a garbage and therefore memory leak

c) Creates a dangling reference

d) Creates aliases

© Confidential Information of PES University. All Rights Reserved - 2026

Revision: Functions

- Associates a sequence of values called arguments with a value the result
- Arguments are copied to the corresponding parameters on a function call
- 'C' supports parameter passing by value
- To change an argument through a parameter, a pointer to the argument need to be passed
- When a function is called, an activation record or stack frame is created and all parameters, local variables, temporary variables, return address are stored on the stack
- When the function returns, the activation record is removed
- On a function call, the controlled is transferred to the beginning of the function
- On return, the instruction following the function call gets executed

© Confidential Information of PES University. All Rights Reserved - 2026

Revision: Functions

- A function calling itself or indirectly is called a recursion
- "if and recursion" is equivalent to an iterative loop
- A recursive function requires "escape hatch" or "base case" to stop recursive calls
- Recursive functions might take more time, but they are easy to develop if the underlying data structure is recursive.

© Confidential Information of PES University. All Rights Reserved - 2026

Revision: Functions

- Divide and conquer
- Sharing
- Maintenance
- Reliability
- Reuse

© Confidential Information of PES University. All Rights Reserved - 2026

Revision: Pointers

- Holds an address of another entity
- is not same as integer
- Has a type ; pointer to int is not same as pointer to double
- Pointer supports 3 arithmetic operation
    - Add an int to a pointer => results in a pointer
    - Subtract an int from a pointer => results in a pointer
    - Take difference between two pointers => results in an int
- Parameter of a function has to be a pointer if the correnspoding argument should be changed by dereferencing

© Confidential Information of PES University. All Rights Reserved - 2026

Revision : Pointers

- Has three points to note
- Aliasing : two pointers point to the same location
    - changing the referent through one also affects referencing through the other
- Dangling pointer : no location; but have access to it
    - Dangling pointers are not dangerous
    - dereferencing a dangling pointer is an undefined behaviour
    - program crashes if you are lucky!
- Garbage : location; but no access to it
    - location is lost until the program terminates
    - so this is called memory leak
    - 'C' does not support garbage collection

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Arrays

Arrays

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Arrays

1. Which of the following is false?

```
int a[10] ;

int main() {
}
```

What can we say about the array a?

a) All elements will be initialized to 0

b) All elements will have the same garbage value

c) Each element will have a distinct garbage value

d) Results in a syntax error

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Arrays

1. Which of the following is false?

```
int a[10] ;

int main() {
}
```

What can we say about the array a?

a) All elements will be initialized to 0

b) All elements will have the same garbage value

c) Each element will have a distinct garbage value

d) Results in a syntax error

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Arrays

2. Comment :

```
int a[] = {10, 20, 30, 40, 50}; int *p = a + 2;

int x = *p++;
```

What happens?

a) x becomes 31; p is not changed

b) x becomes 30; p points to a[3];

c) Syntax error; initialize of p not correct

d) x becomes 40; p points to a[3];

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Arrays

2. Comment :

```
int a[] = {10, 20, 30, 40, 50}; int *p = a + 2;

int x = *p++;
```

What happens?

a) x becomes 31; p is not changed

b) x becomes 30; p points to a[3];

c) Syntax error; initialize of p not correct

d) x becomes 40; p points to a[3];

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Arrays

3. Comment :

```
int a[] = {1, 2, 3}; int b[] = {1, 2, 3};

a == b
```

a) Is a syntax error; arrays cannot be compared

b) 1 as both are arrays contain the same elements

c) 0 as both the arrays point to different locations

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Arrays

3. Comment :

```
int a[] = {1, 2, 3}; int b[] = {1, 2, 3};

a == b
```

a) Is a syntax error; arrays cannot be compared

b) 1 as both are arrays contain the same elements

c) 0 as both the arrays point to different locations

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Functions, Pointers and Arrays

4. Comment :

sizeof int on this implementation is say 4 bytes.

```
int a[] = {10, 20, 30, 40, 50}; int *p = a + 5;
```

what is the value of p – a?

a) 20

b) 5

c) Depends on implementation

d) Error: pointer – array not allowed

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Functions, Pointers and Arrays

4. Comment :

sizeof int on this implementation is say 4 bytes.

```
int a[] = {10, 20, 30, 40, 50}; int *p = a + 5;
```

what is the value of p – a?

a) 20

b) 5

c) Depends on implementation

d) Error: pointer – array not allowed

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Functions

5.

int a[5][4]; The array is arranged linearly in the memory of the computer.

Counting from 0, what is the position of a[3][2] in the array?

a) 14

b) 17

c) 13

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Functions

5.

int a[5][4]; The array is arranged linearly in the memory of the computer.

Counting from 0, what is the position of a[3][2] in the array?

a) 14

b) 17

c) 13

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Revision: Array

- has # of elements
- all the elements should be of the same type - homogeneous
- size of the array is specified at the point of creation
  - size is given as a constant at compile time
  - size could be specified at runtime in C99 VLA : variable length array
- array cannot grow or shrink
- elements are selected based on index or subscript
- index is an integer; starting value is 0 - zero based indexing
- supports random access
  - time required to access the element does not depend on its position
  - is always same

© Confidential Information of PES University. All Rights Reserved - 2026

Revision Array

- Array at compile time is an array. It knows everything about itself.
- Array at runtime degenerates to a pointer.
    - At runtime, the size of the array is not stored.
- Index out of bounds - accessing elements beyond the size of the array cannot be checked.
    - Checking would require the size of the array at runtime - extra memory
    - Checking would incur some runtime cost.
    - This if done would be penalizing the right programmer
- So, 'C' follows this philosophy. The programmer gets what he deserves!
- C array at runtime is a constant pointer
- So, arrays cannot be assigned one to another. Arrays are not assignment compatible.

© Confidential Information of PES University. All Rights Reserved - 2026

Revision: Array

- Every element of an array itself can be an array
- This is called multidimensional array
- Stored in a row major fashion
- Size of each dimension but for the first required to access an element of the multidimensional array
- If a 2 D array is passed as an argument to a function, the corresponding parameter is a pointer to an array
- void foo(int a[][10]); // only 2 D array having 10 columns can be passed as argument
- void bar(int n, int a[][n]); // Any 2 D array can be passed as argument as long as the first argument indicates the number of columns

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Structures

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Structure

```
struct { int a; double b; } x;
```

a) x is a variable having a and b as components

b) x is a type having a and b as components

c) x->a is an int variable

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Structure

```
struct { int a; double b; } x;
```

a) x is a variable having a and b as components

b) x is a type having a and b as components

c) x->a is an int variable

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Structure

```
typedef struct { int a; double b; } x;
```

a) x is a variable having a and b as components

b) x is a type having a and b as components

c) x->a is an int variable

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Structure

```
typedef struct { int a; double b; } x;
```

a) x is a variable having a and b as components

b) x is a type having a and b as components

c) x->a is an int variable

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Structure

```
struct what { char ch; int n; } ;
```

if sizeof int is 4 bytes and char is 1 byte,

sizeof this structure is

a) 5

b) 8

c) Depends on the compiler

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Structure

```
struct what { char ch; int n; } ;
```

if sizeof int is 4 bytes and char is 1 byte,

sizeof this structure is

a) 5

b) 8

c) Depends on the compiler

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Structure

```
struct what { int a[10]; double b; } c[5];
```

How to refer to 5th element of a in the 2nd element of c counting from 0?

a) c[1].a[4]

b) c[1]->a[4]

c) c.a[1][4]

d) C[1][4].a

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Structure

```
struct what { int a[10]; double b; } c[5];
```

How to refer to 5th element of a in the 2nd element of c counting from 0?

a) c[1].a[4]

b) c[1]->a[4]

c) c.a[1][4]

d) C[1][4].a

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Structure

```
struct what { int a; double b; } a = {1279, 3.14}, b;

b = a;
```

a) Syntax error; structures cannot be assigned

b) The variable b will point to a

c) The variable b gets a fieldwise copy of b

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Structure

```
struct what { int a; double b; } a = {1279, 3.14}, b;

b = a;
```

a) Syntax error; structures cannot be assigned

b) The variable b will point to a

c) The variable b gets a fieldwise copy of b

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Structure

- A structure has one or more components
- components are generally heterogeneous
- components have names - structure has named components
    - in case of array, the components are computed based on the value of an expression
- offset to each component is fixed at compile time
- supports random access. Time required to access the component does not depend on its position in the layout of the structure.

© Confidential Information of PES University. All Rights Reserved - 2026

Structure

- The size of a structure depends on implementation.
- The size of structure is at least equal to the sum of the sizes of data members.
- A structure variable stands for the whole structure.
- It does not degenerate to a pointer as is the case with arrays.
- Structures are assignment compatible.
- Conceptually, the compiler generates the code for memberwise copy.
- Structures can be assigned even if the structure contains an array.

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Dynamic Allocation

Dynamic Allocation

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Dynamic Allocation

```
int *p = (int*) malloc(sizeof(int)); *p = 100; int *q = p; int a = *p; *q = 110;
```

What are values of a, *p, *q assuming malloc succeeds?

a) 110 110 110

b) 100 110 110

c) 100 100 110

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Dynamic Allocation

```
int *p = (int*) malloc(sizeof(int)); *p = 100; int *q = p; int a = *p; *q = 110;
```

What are values of a, *p, *q assuming malloc succeeds?

a) 110 110 110

b) 100 110 110

c) 100 100 110

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Dynamic Allocation

```
int *p = (int*) malloc(sizeof(int)); *p = 100; int *q = p;

   free(q); printf("%d ", *p); // assume malloc succeeds?
```

a) 100

b) Cannot call free on q as there was allocation on p

c) Undefined behaviour as p is a dangling pointer

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Dynamic Allocation

```
int *p = (int*) malloc(sizeof(int)); *p = 100; int *q = p;

   free(q); printf("%d ", *p); // assume malloc succeeds?
```

a) 100

b) Cannot call free on q as there was allocation on p

c) Undefined behaviour as p is a dangling pointer

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Dynamic Allocation

```
void foo() {
    int *p = (int*)malloc(sizeof(int)); // assume malloc succeeds
    // some operations here

}
```

a) Clean code as p is removed from the stack at the end of the block

b) Results in Dangling pointer at the end of the block

c) Results in garbage; garbage collector makes it useful again

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Dynamic Allocation

```
void foo() {
    int *p = (int*)malloc(sizeof(int)); // assume malloc succeeds
    // some operations here

}
```

a) Clean code as p is removed from the stack at the end of the block

b) Results in Dangling pointer at the end of the block

c) Results in garbage; garbage collector makes it useful again

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Dynamic Allocation

```
struct node { int key; struct node next; } x;
```

a) Will not compile; a structure cannot have a structure of the same type within it

b) No error; struct will have the combined size of an int and a pointer

c) No error; x.next will be a structure with key and next as the fields

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Dynamic Allocation

```
struct node { int key; struct node next; } x;
```

a) Will not compile; a structure cannot have a structure of the same type within it

b) No error; struct will have the combined size of an int and a pointer

c) No error; x.next will be a structure with key and next as the fields

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Dynamic Allocation

```
struct node { int key; struct node* next; } x;

x.key = 10; x.next = &x;
```

a) Will not compile; x.next and &x are not of the same type

b) x.next.key is 10

c) x.next->next will be a dangling pointer

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Dynamic Allocation

```
struct node { int key; struct node* next; } x;

x.key = 10; x.next = &x;
```

a) Will not compile; x.next and &x are not of the same type

b) x.next.key is 10

c) x.next->next will be a dangling pointer

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Dynamic Allocation

- C does not support any operator for memory allocation or deallocation
- These are realized by using the standard library functions
- void* malloc(int size);
    - Is a low level function
    - Should indicate how many bytes are required
    - Allocates a block of so many bytes and returns a malleable pointer to it
    - Does book keeping – remembers the size of the block allocated
    - You may cast it to any type you want
    - Returns NULL when it fails

© Confidential Information of PES University. All Rights Reserved - 2026

Dynamic Allocation

- void free(void *ptr);
  - Frees the block pointed to by ptr
  - Makes the block of memory useful again
  - Observe that we do not specify the size of the block allocated
  - Uses book keeping information created while allocating
  - Calling free on a pointer value not returned by malloc or calling second time are undefined behaviour
  - Free(NULL) is a no op.

© Confidential Information of PES University. All Rights Reserved - 2026

Dynamic Allocation

- Garbage : location with no access
- Happens only with dynamic allocation
- Happens when memory is allocated and not freed
- Happens when the pointer holding the pointer value to the allocated block is changed or dies
- Garbage becomes memory leak in 'C' as 'C' does not support garbage collector
- All the garbage is collected when the program terminates

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Abstract Data Type (ADT)

ADT

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key = 20; p->next_->next_ = NULL;

p = p->next; // comment
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) Results in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key = 20; p->next_->next_ = NULL;

p = p->next; // comment
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) Results in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

P->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key_ = 20; p->next_->next_ = NULL;
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) Results in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

P->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key_ = 20; p->next_->next_ = NULL;
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) Results in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10; free(p); // p ?
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) p in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10; free(p); // p ?
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) p in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key_ = 20;
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) P->next_->next_ in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = (node_t*)malloc(sizeof(node_t)); p->next_->key_ = 20;
```

a) p->key_ is 10

b) p->next_->key_ is 20

c) P->next_->next_ in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = p;
```

a) Runtime error

b) p->next_->key_ is 10

c) P->next_->next_ in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Dynamic Allocation

```
struct node { int key_; struct node* next_}; typedef struct node node_t;

node_t *p = (node_t*)malloc(sizeof(node_t)); p->key_ = 10;

p->next_ = p;
```

a) Runtime error

b) p->next_->key_ is 10

c) P->next_->next_ in dangling pointer

d) Results in garbage therefore memory leak

© Confidential Information of PES University. All Rights Reserved - 2026

Concept of Type

- An integer in Mathematics is a set
- t is closed for addition and multiplication
- 25 + 34 is 59
- How do we add?
    - Add unit digits and then digits in the tenth place and so on along with the carry from the earlier stage
    - Add row wise
    - Use fingers and toes
    - Use tally marks
    - Convert to some other base, add and convert the result back to decimal
- Will the result be different?
- Should we learn how to add if the integers have different values?

© Confidential Information of PES University. All Rights Reserved - 2026

Concept of Type

- An integer is a type
- Represented in a programming language normally as a subset
- A type indicates
    - What values are legal
    - What we can do with them
    - Does not say how we can do them
    - Allows us treat almost all values of that type uniformly
- Type of a variable in 'C' has to be declared in the beginning
- Once declared, cannot be changed.
- Type of a variable is determined at compile time
- In Python,
    - Type is associated with a value.
    - A variable gets a type based on type of value assigned.
    - Type is dynamic.

© Confidential Information of PES University. All Rights Reserved - 2026

Type and a programming language

- Can a language support all possible types of the real world?
- A car is a type.
- Supports the following operations
  - Start
  - Stop
  - Accelerate
  - Brake
  - Steer
- Once you have learnt driving a car, you can (almost) drive any other car
- Other types of real world : fan, mobile phone, browser, laptop, novel, dream, ...
- Can a language support all possible types of the real world?
- NO

© Confidential Information of PES University. All Rights Reserved - 2026

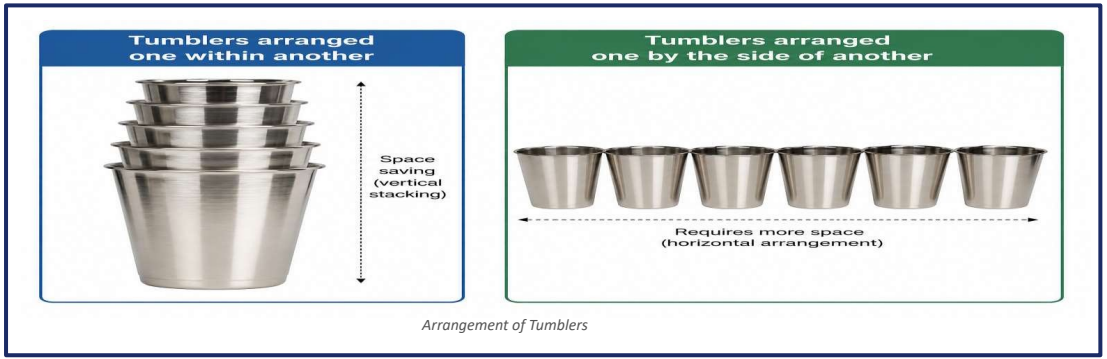
Type and a programming language

- A Language should support a way of creating our own type
- Abstract Data Type (ADT)
    - Like a type (int, double, ...) in a language, this supports
        - A few operations
        - Specifies what they do and not how they do
        - Specifies the interface and not the implementation
        - Both the user and the implementor depend on the interface
        - Interface once decided is sacred; cannot be changed
        - Implementation can be changed without affecting the user
        - This is called inversion of dependency
- If B depends on A, any change to A would require change to B.
- If both A and B depend on what does not change, change to A will not affect B

© Confidential Information of PES University. All Rights Reserved - 2026

Data Structure

- A data structure is an abstract data type
- Has number of components arranged in some suitable manner
    - To achieve some goal
- An example: arrangement of Tumblers



Arrangement of Tumblers

© Confidential Information of PES University. All Rights Reserved - 2026

Data Structure



Another Example: Arrangement of Books

© Confidential Information of PES University. All Rights Reserved - 2026

List

- Is a data structure
- Has zero or more components
- Each element has an ordinal position
- Cannot directly access the element based on position
    - Sequential access
    - No random or direct access
- Insertion and deletion once the position is known requires
    - A few operations

© Confidential Information of PES University. All Rights Reserved - 2026

List

- Interface :
  - Create
  - Destroy
  - Insert
  - Delete
  - ...
- Implement using
  - A linked list
  - An Array

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

Linked List

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: ADT and List

An Abstract Data type specifies

a) Both interface and implementation

b) Only Implementation

c) Only interface

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: ADT and List

An Abstract Data type specifies

a) Both interface and implementation

b) Only Implementation

c) Only interface

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: ADT and List

An array is a data structure in which

a) Access is faster and insertion/deletion are not

b) Both access and insertion/deletion are fast

c) Both access and insertion/deletion are slow

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: ADT and List

An array is a data structure in which

a) Access is faster and insertion/deletion are not

b) Both access and insertion/deletion are fast

c) Both access and insertion/deletion are slow

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: ADT and List

A list is a data structure in which

a) Access is faster and insertion/deletion are not

b) Both access and insertion/deletion are fast

c) Both access and insertion/deletion are slow

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: ADT and List

A list is a data structure in which

a) Access is faster and insertion/deletion are not

b) Both access and insertion/deletion are fast

c) Both access and insertion/deletion are slow

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: ADT and List

What is true of a list?

a) Can access the ith element directly

b) Cannot be empty

c) Every element has a position

d) Can grow indefinitely

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: ADT and List

What is true of a list?

a) Can access the ith element directly

b) Cannot be empty

c) Every element has a position

d) Can grow indefinitely

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: ADT and List

Which of the following on a list always requires less operations

a) Add in the beginning

b) Add in the middle

c) Find the midpoint

d) Remove an element at the end

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: ADT and List

Which of the following on a list always requires less operations

a) Add in the beginning

b) Add in the middle

c) Find the midpoint

d) Remove an element at the end

© Confidential Information of PES University. All Rights Reserved - 2026

Evolving a Linked List

- Self referential data structure :
  - Is a structure where a field refers to the same structure type
  - Is implemented as a pointer to the same structure

```
struct node
{
    int key_;
    struct node *next_;
};
typedef struct node node_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Evolving a Linked List

What happens here?

```
node_t *p;
p = (node_t*)malloc(sizeof(node_t));
p->key_ = 20;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Evolving a Linked List

How about now?

```
node_t *p;
p = (node_t*)malloc(sizeof(node_t));
p->key_ = 20;

p->next_ = (node_t*)malloc(sizeof(node_t));
p->next_->key_ = 40;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Evolving a Linked List

Some more code?

```
node_t *p;
p = (node_t*)malloc(sizeof(node_t));
p->key_ = 20;

p->next_ =
(node_t*)malloc(sizeof(node_t));
p->next_->key_ = 40;

p->next_->next_ =
(node_t*)malloc(sizeof(node_t));
p->next_->next_->key_ = 60;
p->next_->next_->next_ = NULL;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Evolving a Linked List

```
void disp(node_t* q)
{
    while(q != NULL)
    {
        printf("%d ", q->key_);
        q = q->next_;
    }
}

disp(p);
```

What is this code? q = q->next_;

Why not q++?

So, We have engineered a linked list!

© Confidential Information of PES University. All Rights Reserved - 2026

A list is either empty or has a node followed by a list

The nodes are accessed starting from the head of the list

The access is sequential.

In the worst case, we may have to traverse all the nodes before reaching the last node. This may require n operations if there are n nodes.

An ordered list is a list where the nodes are arranged in the order of keys.

Nodes should be inserted based on the comparison of the keys.

Nodes can be inserted in an empty list or in the beginning or the middle or the end.

© Confidential Information of PES University. All Rights Reserved - 2026

Structure of list :

- The structure of node is similar to what we discussed before.
  - A key field and a pointer field to provide for self reference

```
struct node
{
    int key_;
    struct node *next_;
};
typedef struct node node_t;
```

- List has a head which will point to the first node in the list.

```
struct mylist
{
    node_t *head_;
};

typedef struct mylist mylist_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Interface : Similar to the last example

- We require functions for creating and destroying the list.
- We require functions to insert into the list in the right position.
- We will also provide a way of traversing and displaying the list.

- void init_list(mylist_t*);
- void insert_list(mylist_t*, int key);
- void disp_list(mylist_t*);
- void free_list(mylist_t*);

© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Initially list is empty. The head_ of the list is grounded.

```
void init_list(mylist_t* ptr_list)
{
    ptr_list->head_ = NULL;
}
```

- This function displays all the keys in the list starting from the head_.

```
void disp_(node_t* q)
{
    while(q != NULL)
    {
        printf("%d ", q->key_);
        q = q->next_;
    }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Clear the list
  - Observe the code in the while.
  - Can we change the order of these statements?

```
void free_list_(node_t* q)
{
    node_t* r;
    while(q != NULL)
    {
        r = q->next_;
        free(q);
        q = r;
    }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Insertion :
- Make a new node; populate it; insert
- Boundary conditions :
  - Empty list
  - Add in the beginning
  - Add in the middle
  - Add at the end

```
// 1.empty list :
// head_ is NULL in the beginning
if(ptr_list->head_ == NULL)
{
    ptr_list->head_ = temp;
    temp->next_ = NULL;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Insertion :
  - Add in the beginning
  - Add in the middle
  - Add at the end
- Traverse the list

```
node_t* pres = ptr_list->head_;
while(pres != NULL && pres->key_ < temp->key_ )
{
    pres = pres->next_;
}
```

- Cannot traverse back
- Solution :
  - One trailing pointer or
  - Look ahead

© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Insertion :
- Traverse the list : use a trailing pointer

```
node_t* prev = NULL;
node_t* pres = ptr_list->head_;
while(pres != NULL && pres->key_ < temp->key_ )
{
  prev = pres;
  pres = pres->next_;
}
```

- prev tags behind the pres
- temp is to be inserted between the nodes to which prev and pres point to.
- What if prev is NULL => insert in the beginning
- Check : insert in the middle or end will have similar code

© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Insertion :

```
if(prev == NULL) // insert in the beginning
{
  temp->next_ = pres;
  ptr_list->head_ = temp;
}
else // middle or end
{
  temp->next_ = pres;
  prev->next_ = temp;
}
```

We have successfully engineered an ordered list!!

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

List with Header Node

List with Header Node

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: List

The list_t has a field called count_ which tracks the number of components in the list.

This implies:

a) This field should be updated on insertion into and deletion of a component from the list

b) The # of operations to find the number of components will still require traversal of the whole list

c) This change is not allowed as it affect the client.

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: List

The list_t has a field called count_ which tracks the number of components in the list.

This implies:

a) This field should be updated on insertion into and deletion of a component from the list

b) The # of operations to find the number of components will still require traversal of the whole list

c) This change is not allowed as it affects the client.

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: List

Adding at the end of the list

a) May require a few operations not proportional to the length of the list

b) Always requires a number of operations proportional to the length of the list

c) Can never support such an operation

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: List

Adding at the end of the list

a) May require a few operations not proportional to the length of the list

b) Always requires a number of operations proportional to the length of the list

c) Can never support such an operation

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: List

```
void what(list_t *ptr_list) { free(ptr_list->head_); }
```

a) This will free all the nodes in the list

b) This will free all but the first node in the list

c) This will free the first node; rest of the list is accessible

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: List

void what(list_t *ptr_list) { free(ptr_list->head_); }

a) This will free all the nodes in the list

b) This will free all but the first node in the list

c) This will free the first node; rest of the list is accessible

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: List

The header node is a dummy node added in the beginning of the list.

a) This makes insertion simpler

b) This makes deletion simpler

c) This makes searching simpler

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: List

The header node is a dummy node added in the beginning of the list.

a) This makes insertion simpler

b) This makes deletion simpler

c) This makes searching simpler

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: List

The tailer node is a dummy node added at the end of the list.

a) This makes insertion simpler

b) This makes deletion simpler

c) This makes searching simpler

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: List

The tailer node is a dummy node added at the end of the list.

a) This makes insertion simpler

b) This makes deletion simpler

c) This makes searching simpler

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

List with Header Node

List

- Is a data structure
- Is an abstract data type
- Has components at an ordinal position
- Two points to note
  - Sequential access
  - Insertion / deletion requires a few operations
- Can implement a list
  - Using linked list
  - Using an array – not discussed
- List can be ordered based on some key

© Confidential Information of PES University. All Rights Reserved - 2026

List with Header Node

List

- Can we change the list implementation without changing the list interface?
- Can we modify the list without affecting the client?
- Add a dummy node in the beginning
- Called the header node
- Will make the insertion simple

© Confidential Information of PES University. All Rights Reserved - 2026

List with Header Node

List

- Initially the head of the list will point to a dummy node
- What should be the value of the key_?

© Confidential Information of PES University. All Rights Reserved - 2026

List with Header Node

List

- Display function will skip over the first and then display the remaining nodes

© Confidential Information of PES University. All Rights Reserved - 2026

List with Header Node

List

- Freeing the list will remove all but the first node

© Confidential Information of PES University. All Rights Reserved - 2026

List with Header Node

List

- Insertion based on the key
  - No empty list ; head_ does not have to change

© Confidential Information of PES University. All Rights Reserved - 2026

List with Header Node

List

- Insertion based on the key
  - No insertion in the beginning; head_ does not have to change

© Confidential Information of PES University. All Rights Reserved - 2026

List with Header Node

List

- Insertion based on the key
  - Insertion in the middle or the end have exactly the same code
  - No selection required once position found

© Confidential Information of PES University. All Rights Reserved - 2026

List with Header Node

List

- With the header node, insertion code has become simpler
- Client is not affected
- Implementation changed independent of the client

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

Professor

Department of Computer Science & Engineering (AI & ML)

Stack

Data Structures and its Applications

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Stack

A stack is a data structure which does not support the following function

a) push

b) pop

c) Is_empty

d) indexing

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Stack

A stack is a data structure which does not support the following function

a) push

b) pop

c) Is_empty

d) indexing

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Stack

If a string is of the form wcw^R, where w is a string of characters not having c and w^R is the reverse of W, then it is a palindrome. To check for a such a palindrome, the size of the stack should be at least

a) The length of w

b) The length of wcw^R

c) The length of wc

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Stack

If a string is of the form wcw^R, where w is a string of characters not having c and w^R is the reverse of W, then it is a palindrome. To check for a such a palindrome, the size of the stack should be at least

a) The length of w

b) The length of wcw^R

c) The length of wc

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Stack

To check whether a given string is of the form $a^n b^{2n}$, some number of a followed by twice the number of b, each time we get a

a) push a symbol into the stack

b) Push 2 symbols into the stack

c) Pop a symbol from the stack

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Stack

To check whether a given string is of the form $a^n b^{2n}$, some number of a followed by twice the number of b, each time we get a

a) push a symbol into the stack

b) Push 2 symbols into the stack

c) Pop a symbol from the stack

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Stack

In an array implementation of the stack, push should

a) Increment top and then use that the index

b) Use top as the index and then increase top

c) Use top and should not change it

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Stack

In an array implementation of the stack, push should

a) Increment top and then use that the index

b) Use top as the index and then increase top

c) Use top and should not change it

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Stack

In an array implementation of the stack, if is_full returns true

a) The program should be aborted

b) The top element should be popped off

c) The stack should be reset

d) The user should decide what is the best action

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Stack

In an array implementation of the stack, if is_full returns true

a) The program should be aborted

b) The top element should be popped off

c) The stack should be reset

d) The user should decide what is the best action

© Confidential Information of PES University. All Rights Reserved - 2026

Stack : Abstract Data Type

stack

- Is a data structure
- Has zero or more components
- Can operate at only one end – normally called the stack top
- Supports LIFO : Last In First Out
- What goes in last, comes out first
- Does not support the concept of traversal

© Confidential Information of PES University. All Rights Reserved - 2026

Stack : Interface

stack

- Create
- Destroy
- Push : add an element at the top
- Pop : remove an element at the top
- Peek : check the top element without removing it
- Is_empty : check whether the stack is empty
- Is_full : check whether the stack is full

© Confidential Information of PES University. All Rights Reserved - 2026

Stack : Implementation

stack

- using
  - A linked list
  - An Array

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Stack Application

Data Structures and its Applications

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Stack Application

How do we make out that (()))) is not well formed in our algorithm?

a) By counting and comparing the number of left and the number of right parentheses

b) By encountering popping from an empty stack

c) By having a non-empty stack when the input string has been completed processed

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Stack Application

How do we make out that (()))) is not well formed in our algorithm?

a) By counting and comparing the number of left and the number of right parentheses

b) By encountering popping from an empty stack

c) By having a non-empty stack when the input string has been completed processed

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Stack Application

How do we make out that ((()) is not well formed in our algorithm?

a) By counting and comparing the number of left and the number of right parentheses

b) By encountering popping from an empty stack

c) By having a non-empty stack when the input string has been completed processed

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Stack Application

How do we make out that ((()) is not well formed in our algorithm?

a) By counting and comparing the number of left and the number of right parentheses

b) By encountering popping from an empty stack

c) By having a non-empty stack when the input string has been completed processed

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Stack Application

On encountering a left parenthesis,

a) Pop the top element from the stack

b) Push some marker onto the stack

c) Should push) only on the stack

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Stack Application

On encountering a left parenthesis,

a) Pop the top element from the stack

b) Push some marker onto the stack

c) Should push) only on the stack

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Stack Application

When an expression with 3 different type of brackets is to be checked for well formedness,

a) We require 6 different types of symbols to put on the stack

b) We require one symbol to put on the stack

c) We require 3 different types of symbols to put on the stack

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Stack Application

When an expression with 3 different type of brackets is to be checked for well formedness,

a) We require 6 different types of symbols to put on the stack

b) We require one symbol to put on the stack

c) We require 3 different types of symbols to put on the stack

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Stack Application

If at some point in processing a bracketed expression, if the stack is empty

a) It means the expression is not well formed

b) It means the expression has had no parentheses until this point

c) It definitely means the expression is well formed no matter what follows.

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Stack Application

If at some point in processing a bracketed expression, if the stack is empty

a) It means the expression is not well formed

b) It means the expression has had no parentheses until this point

c) It definitely means the expression is well formed no matter what follows.

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Stack : Applications

stack

- All stack operations happen at the stack top.
- Push adds an element to the top.
- Pop removes the top element.
- Peek inspects the top element.
- The implementation may use an array or a linked list
- The interface is what the user needs to know.

© Confidential Information of PES University. All Rights Reserved - 2026

Stack : Applications

Stack : Well formed parentheses

- Why Counting Parentheses Is Not Enough
- Examples: (3 + 4) and ((5 + 6) * (7 + 8)) are well formed.
- Ignoring other characters gives () and (()()).
- Equal numbers of '(' and ')' are necessary, but not sufficient.
- Example:)(is invalid because a right parenthesis occurs before a left one.
- Example: ())(()) is also invalid

© Confidential Information of PES University. All Rights Reserved - 2026

Stack : Applications

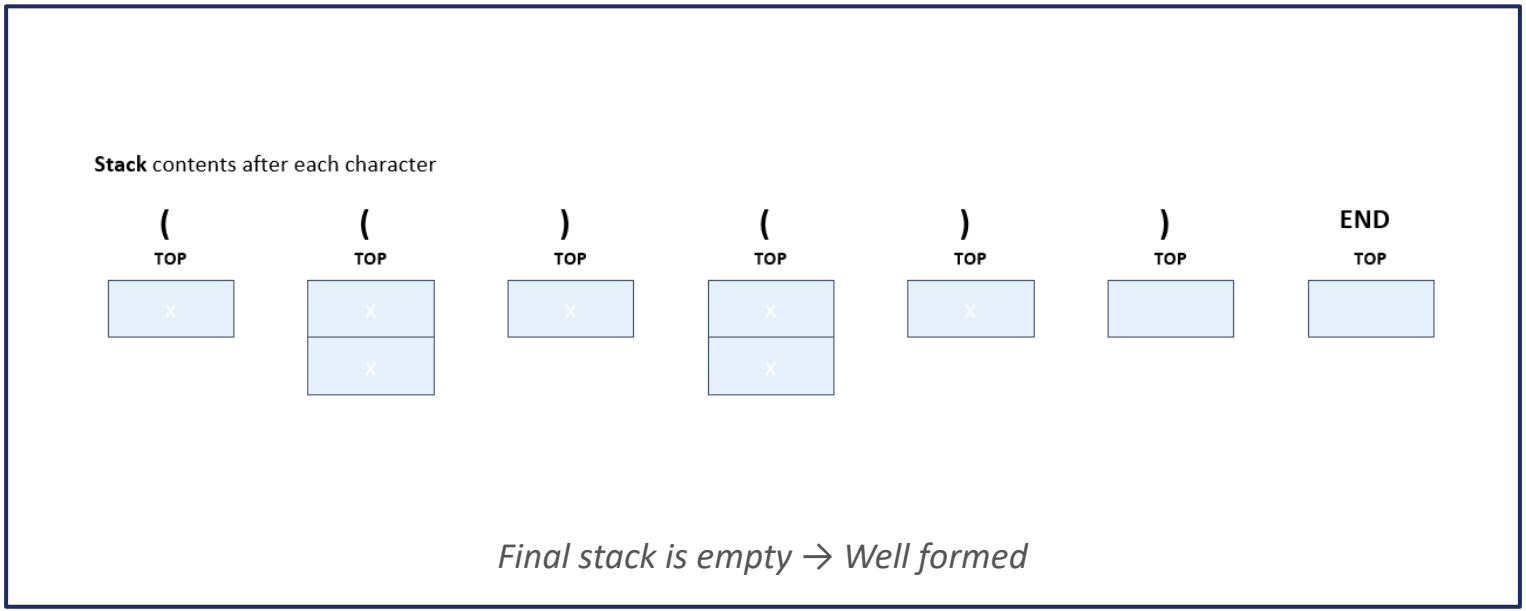
Algorithm: Well-Formed Parentheses

1. Initialize the stack.
2. Read the expression as a string and scan character by character.
3. If the character is '(', push a placeholder.
4. If the character is ')', pop; if the stack is empty, the expression is invalid.
5. After processing the expression, the stack must be empty.

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application

Example: Checking (()())



© Confidential Information of PES University. All Rights Reserved - 2026

Algorithm: Well-Formed Parentheses : C Code

```
int is_valid = 1;
char ch; char *s = str;
while (is_valid && (ch = *s++) != '\0') {
    if (ch == '(')
        push(&mystack, 'x');
    else if (ch == ')') {
        if (!is_empty(&mystack))
            pop(&mystack);
        else
            is_valid = 0;
    }
}
```

```
if (is_valid && is_empty(&mystack))
    printf("Well formed expression");
else
    printf("not well formed expression");
```


© Confidential Information of PES University. All Rights Reserved - 2026

Stack : Applications

Algorithm: Well-Formed Brackets: (), [], {}

- With multiple bracket types, a generic placeholder is not enough.
- For '(' push ')'; for '[' push ']'; for '{' push '}'.
- When a closing bracket is encountered, the stack must not be empty.
- The closing bracket must match the character at the top of the stack.
- Well formed: (){}[], ([]{}) and ([{}]).
- Not well formed: ([)], ([]]) and (][).

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Stack Application: Expression Conversion and Evaluation

Data Structures and its Applications

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Stack Application: expression conversion and evaluation

a = b = c

The equivalent expression in postfix is

a) a b = c =

b) a b c = =

c) c b = a =

d) c b a = =

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Stack Application: expression conversion and evaluation

a = b = c

The equivalent expression in postfix is

a) a b = c =

b) a b c = =

c) c b = a =

d) c b a = =

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Stack Application: expression conversion and evaluation

While evaluating an expression in postfix, we require

a) a stack of operators

b) a stack of operands

c) Stacks of operator and operands

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Stack Application: expression conversion and evaluation

While evaluating an expression in postfix, we require

a) a stack of operators

b) a stack of operands

c) Stacks of operator and operands

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Stack Application: expression conversion and evaluation

In a postfix expression,

a) Left parenthesis has the highest precedence

b) Right parenthesis has the highest precedence

c) There are no parentheses

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Stack Application: expression conversion and evaluation

In a postfix expression,

a) Left parenthesis has the highest precedence

b) Right parenthesis has the highest precedence

c) There are no parentheses

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Stack Application: expression conversion and evaluation

While converting an expression from infix to postfix,

a) Left parenthesis has the highest precedence in the input string

b) Left parenthesis has the highest precedence in the stack

c) Precedence of parentheses is same in the input string as well as in the stack

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Stack Application: expression conversion and evaluation

While converting an expression from infix to postfix,

a) Left parenthesis has the highest precedence in the input string

b) Left parenthesis has the highest precedence in the stack

c) Precedence of parentheses is same in the input string as well as in the stack

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Stack Application: expression conversion and evaluation

For this postfix expression $789 + *$, the corresponding infix expression is

a) $7 * 8 + 9$

b) $7 + 8 * 9$

c) $(7 + 8) * 9$

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Stack Application: expression conversion and evaluation

For this postfix expression $789 + *$, the corresponding infix expression is

a) $7 * 8 + 9$

b) $7 + 8 * 9$

c) $(7 + 8) * 9$

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

stack

- Arithmetic expressions are easy for humans to write
- parsing them in a program can be complicated.
- Infix notation uses operators between operands: 3 + 4.
- Postfix notation puts operators after operands: 3 4 +.
- A stack makes postfix evaluation straightforward.

© Confidential Information of PES University. All Rights Reserved - 2026

Why Expression Evaluation Is Complicated

- BODMAS gives a familiar order: Brackets, Orders, Division, Multiplication, Addition, Subtraction.
- Programming languages add many precedence levels and associativity rules.
- Parentheses introduce nesting.
- A useful alternative is a notation in which operators can be processed as soon as they are encountered.

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Infix and postfix expressions

- 3 + 4 => 3 4 +
- 5 * 6 + 7 => 5 6 * 7 +
- 5 + 6 * 7 => 5 6 7 * +
- (3 + 4) * (5 + 6) => 3 4 + 5 6 + *
- Observe : no parentheses in postfix expression

© Confidential Information of PES University. All Rights Reserved - 2026

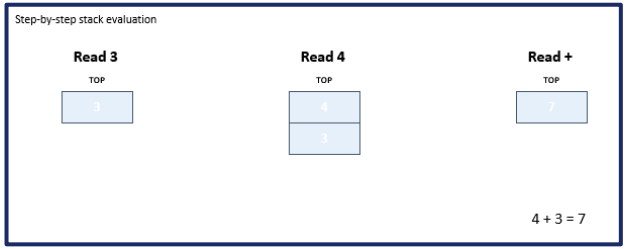
Postfix Evaluation Using a Stack

1. Create a stack of operands.
2. Read the postfix expression from left to right.
3. Operand → push it onto the stack.
4. Operator → pop the required operands, perform the operation, and push the result.
5. After the complete expression is processed, the stack contains exactly one value: the result.

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Example: 3 4 +

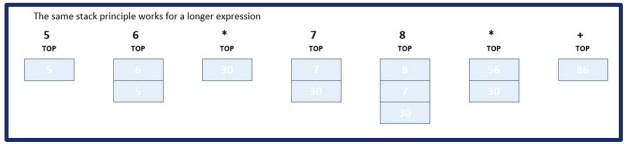


Final result = 7

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Example: 5 6 7 8 +



30 + 56 = 86 → Final result = 86

© Confidential Information of PES University. All Rights Reserved - 2026

Postfix Evaluation: Algorithm

Read the postfix expression.

Create and initialize a stack of integers.

```
For each token:
if token is an operand:
push(token)
else if token is an operator:
val2 = pop()
val1 = pop()
result = val1 operator val2
push(result)
```

```
At the end:
result = pop()
the stack should now be empty.
```


© Confidential Information of PES University. All Rights Reserved - 2026

Postfix Evaluation: order of evaluation

For + and *, operand order does not affect the result.

For – and /, it matters.

If the postfix expression is: 8 3 –

First pop gives val2 = 3; second pop gives val1 = 8.

Compute val1 – val2 = 8 – 3 = 5.

Never compute 3 – 8

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Conversion from infix to postfix

- Example: 5 * 6 + 7 * 8
- Postfix form: 5 6 * 7 8 * +
- Operands are sent directly to the output.
- Relative positions of the operands is never changed
- A stack of operators is required
- Operators are temporarily stored on a stack.
- The stack allows operators to be delayed

until their precedence requires them to be output

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Conversion from infix to postfix

- Associate a precedence value with each operator
- Higher the precedence, higher the value
- When an operator is encountered, we compare the precedence value of the operator in the string with that of the operator in the top of the stack
- While the first one is higher, push the operator into the stack
- Otherwise pop the operator from the stack and push the operator

in the string on to the stack

- When the input string is completely processed, pop all the operators from the stack

© Confidential Information of PES University. All Rights Reserved - 2026

Conversion from infix to postfix: Parentheses During Infix → Postfix

A left parenthesis is always pushed onto the operator stack.

It acts as a barrier: operators inside the parentheses are handled before operators outside.

When a right parenthesis is encountered, pop operators until the matching left parenthesis is found.

The parentheses themselves are not included in the postfix output.

Example: $3 * (4 + 5) * 6 \rightarrow 345 + * 6 *$.

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Summary

- Postfix notation removes the need for precedence rules during evaluation.
- A stack of operands is sufficient for postfix evaluation.
- Operators pop operands, compute, and push the result.
- Infix-to-postfix conversion uses a stack of operators.
- Precedence, associativity, and parentheses determine when operators are popped.
- The stack can be used through its interface without knowing whether it is implemented with an array or a list.

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)

Experiential Learning

polynomial

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Experiential Learning

Polynomial : create a polynomial

- Task 1
  - Input : ordered list of integer – both interface and implementation
  - ToDo :
    - Create a type called Term : term_t
      - Has coefficient and exponent
      - Develop set and display functions
      - Develop function to compare the terms based on the exponent
    - Modify the list of int to list of terms in the decreasing order of exponent

© Confidential Information of PES University. All Rights Reserved - 2026

Experiential Learning

Polynomial : Evaluate a polynomial

- Task 2
  - Input : ordered list of terms – both interface and implementation
  - ToDo :
    - Add a function to evaluate the polynomial given the value of the variable

© Confidential Information of PES University. All Rights Reserved - 2026

Experiential Learning

Polynomial : copy the polynomial

- Task 3
  - Input : ordered list of terms – both interface and implementation
  - ToDo :
    - Create a source list
    - Populate it
    - Create a destination list
    - Write a function to copy the source list to the destination list

© Confidential Information of PES University. All Rights Reserved - 2026

Experiential Learning

Polynomial : differentiate the polynomial

- Task 4
  - Input : ordered list of terms – both interface and implementation
  - ToDo :
    - Create a source list
    - Populate it
    - Create a destination list
    - Differentiate the terms in the source list and add them to the destination list

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

Professor

Department of Computer Science & Engineering (AI & ML)

Introduction

Skiplist and Multilist: Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Skiplist and Multilist

1 What is true in a skiplist?

a) There are multilevels, each of which has the same number of nodes
b) The number of pointers in each node depends on the number of levels planned
c) The traversal for any operation starts at the higher level and this level has all the nodes
d) Any insertion will affect atmost one level

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Skiplist and Multilist

1 What is true in a skiplist?

a) There are multilevels, each of which has the same number of nodes

b) The number of pointers in each node depends on the number of levels planned

c) The traversal for any operation starts at the higher level and this level has all the nodes

d) Any insertion will affect atmost one level

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Skiplist and Multilist

2 What is true in a multilist?

a) This is same as a multidimensional array
b) Every node will have only single pointer
c) Every node may participate in multiple lists
d) Addition of a node would require updation in only one list

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Skiplist and Multilist

2 What is true in a multilist?

a) This is same as a multidimensional array

b) Every node will have only single pointer

c) Every node may participate in multiple lists

d) Addition of a node would require updation in only one list

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Skiplist and Multilist

3 sparse matrix is an example of

a) Using more space to gain in time

b) Using less space at the cost of time

c) a 2D array which can be directly indexed

d) A matrix where most of the elements are non zero

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Skiplist and Multilist

3 sparse matrix is an example of

a) Using more space to gain in time

b) Using less space at the cost of time

c) a 2D array which can be directly indexed

d) A matrix where most of the elements are non zero

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Skiplist and Multilist

4 In a SkipList, the bottom-most level contains:

a) Only the largest element

b) Only the smallest element

c) All the elements in sorted order

d) Randomly selected elements

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Skiplist and Multilist

4 In a SkipList, the bottom-most level contains:

a) Only the largest element

b) Only the smallest element

c) All the elements in sorted order

d) Randomly selected elements

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Skiplist and Multilist

5. A multilist is particularly useful when:

a) Data needs to be accessed through multiple relationships

b) Only sequential access is required

c) Memory must not contain pointers

d) Only one ordering of data is needed

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Skiplist and Multilist

5. A multilist is particularly useful when:

a) Data needs to be accessed through multiple relationships

b) Only sequential access is required

c) Memory must not contain pointers

d) Only one ordering of data is needed

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist and Multilist

ADT: Revision

- Lists and stacks can be implemented in different ways while supporting the same interface.
- Separating concerns leads to good software development.
- A data structure has components; each component should have its own behaviour.
- Example: a polynomial can be represented using an array or a list of terms.

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist and Multilist

Skiplist: Intuition

- A normal shuttle bus stops at every village and may take a long time.
- An express bus can skip villages and then a local bus can reach the destination.
- This idea of skipping portions of a sorted list is called a skip list.

© Confidential Information of PES University. All Rights Reserved - 2026

Limitation of a Normal List

- Example: 10 → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → 100
- Searching for 90 requires traversing almost the whole list.
- Worst-case steps and comparisons are proportional to the number of elements

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist and Multilist

Structure of a Skiplist

- Level 2: 10 ——————→ 40 ——————→ 70 ———→ 100
- Level 1: 10 ——→ 20 ——→ 40 ——→ 50 ——→ 70 ——→ 100
- Level 0: 10 → 20 → 30 → 40 → 50 → 60 → 70 → 80 → 90 → 100
- Higher levels provide shortcuts;
- level 0 walks through the list one step at a time.

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist and Multilist

Searching for 90 in a Skiplist

- Start at 10 on the highest level.
- Move to 40, then 70 because each is less than 90.
- The next node is 100, which is greater than 90.
- Drop down one level at 70.
- Drop one more level at 70.
- At level 0: 70 → 80 → 90; reach 90

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist and Multilist

Searching for 90 in a Skiplist

- Start at 10 on the highest level.
- Move to 40, then 70 because each is less than 90.
- The next node is 100, which is greater than 90.
- Drop down a level at 70.
- At level 0: 70 → 80 → 90; reach 90

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist and Multilist

Why the Skiplist Helps?

- Higher levels allow us to jump over large portions of the list.
- Instead of examining all eight elements before 90, only a few nodes are examined.
- A skip list is a sorted linked list augmented with multiple levels of forward pointers.

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist: Structure and Operations

- A skip list may have any number of levels.
- The number of levels depends on the expected number of nodes.
- Normally, each higher level has about half the nodes of the next level.
- Each node has active pointers for the levels in which it participates. Other pointers are grounded.
- Insertion and deletion follow the same top-down idea as searching.

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist: Applications and Salient Points

- In-memory ordered data for faster insertion, deletion and searching.
- Concurrent data structures where operations can happen in parallel.
- Priority ordered events.
- Can be used instead of balanced search trees.
- Bottom level contains all elements; higher levels contain fewer elements and provide shortcuts.
- Note: Implementation of a skiplist is not part of the source's stated syllabus.

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist and Multilist

Multilists

- A multilist means a list of lists.
- A node may be part of more than one list.
- A list may have nodes whose keys are heads of another list.
- The same node can be reached through different linked lists representing different relationships.

© Confidential Information of PES University. All Rights Reserved - 2026

Sparse Matrix: Motivation

- Very large matrices may contain mostly zero elements.
- Storing every element consumes a lot of memory.
- Store only non-zero elements together with their row and column information.
- For efficient rowwise and columnwise traversal, maintain a row list for each row and a column list for each column.

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist and Multilist

One Physical Node, Multiple Logical Views

- Example: students take courses, and each course has students.
- One view finds the courses taken by a student.
- Another view finds the students who have taken a course.
- Keeping separate structures would duplicate data and require multiple updates.
- A multilist captures the mapping and supports both traversals.

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist and Multilist

Applications of Multilists

- Sparse matrix
- Graph representation
- Social networks
- Multiple views as in files

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist and Multilist

Multilist: Salient Points

- Supports Multiple relations
- Avoids duplicates
- Avoids multiple updation
- Has Complicated pointer management
- Has Complicated insertion and deletion

© Confidential Information of PES University. All Rights Reserved - 2026

Skiplist and Multilist

Summary

- Skiplist: a sorted linked list with multiple levels of forward pointers.
- Higher skiplist levels provide shortcuts for faster searching.
- Multilist: a list of lists in which a physical node can participate in multiple logical lists.
- Multilists are useful when the same data needs multiple views or relationships.

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)